

FUNDSROOM ERP - Technical Documentation

1. Architecture Overview

The FUNDSROOM ERP system is built using a modern decoupled architecture:

Frontend Layer

- **Framework:** React.js with Vite
- **State Management:** Zustand (for global authentication and app state)
- **Styling:** Tailwind CSS for responsive and modern UI
- **Routing:** React Router DOM (Single Page Application)
- **Deployment:** Configured for Vercel / Cloudflare Pages

Backend Layer

- **Runtime:** Node.js
- **Framework:** Express.js
- **Architecture:** RESTful API design
- **Database:** PostgreSQL (Serverless via Neon DB)
- **Media Storage:** Cloudinary (for product images)
- **Deployment:** Configured for Render

Communication

- The frontend communicates with the backend via stateless REST APIs.
- Authentication is handled via JSON Web Tokens (JWT) sent via Authorization headers.
- File uploads are managed via Multer and streamed directly to Cloudinary.

2. API Documentation

Authentication (/api/auth)

- **POST /login** - Authenticates user and returns JWT token.
- **POST /register** - Registers a new user.

Customers (/api/customers)

- **GET /** - Fetch all customers.
- **POST /** - Add a new customer.
- **PUT /:id** - Update existing customer details.

Products & Inventory (/api/products)

- **GET /** - Fetch all products with current stock.
- **POST /** - Add a new product.
- **PUT /:id** - Update a product.
- **POST /:id/image** - Upload product image to Cloudinary (multipart/form-data).
- **POST /:id/stock** - Adjust stock manually (IN/OUT).

Sales Orders (/api/sales-orders)

- **GET /** - Fetch all sales orders.
- **POST /** - Create a new sales order (reserves stock).
- **PUT /:id/status** - Update order status (Created, Reserved, Dispatched, Delivered).

Challans (/api/challans)

- **GET /** - Fetch all delivery challans.
- **POST /** - Create a new manual challan and deduct stock.
- **POST /from-order/:sales_order_id** - Generate challan directly from a Sales Order.
- **PUT /:id/status** - Update challan status.

Invoices & Payments (/api/invoices, /api/payments)

- **GET /invoices** - Fetch all invoices.
- **POST /invoices** - Generate an invoice.
- **POST /payments** - Log a payment against an invoice and update outstanding balance.

3. Known Limitations & Incomplete Parts

1. **Email Service Dependency:** The application utilizes `nodemailer` with `Ethereal` email for testing. In production, this needs to be swapped to a real SMTP provider (e.g., SendGrid, AWS SES) or emails will not be delivered to actual users.
2. **Pagination:** Some API endpoints (like Customers and Challans) fetch all records. As the database grows, explicit `offset/limit` pagination needs to be fully implemented on both the frontend tables and backend queries to maintain performance.
3. **No PDF Generation in Browser:** Currently, reports are exported as CSVs. Generating native PDF invoices from the frontend is not yet implemented.
4. **Data Soft Deletion:** Records are currently hard-deleted via CASCADE constraints or not deleted at all. A soft-delete implementation (e.g., `is_deleted = true`) is recommended for historical auditing.
5. **Real-time Updates:** The chat module and stock updates require a page refresh or manual fetch. WebSockets (Socket.io) could be implemented for real-time live updates across the warehouse and sales floors.